

MÓDULO I · INGENIERÍA DE IA

04

CAPÍTULO

*Agentes de IA: del procesamiento del
lenguaje a la ejecución de tareas*

Agentes de IA: del procesamiento del lenguaje a la ejecución de tareas

"Un modelo responde preguntas. Un agente persigue objetivos."

Objetivos de aprendizaje

Al finalizar esta sección podrás:

- Comprender qué es un agente de IA desde una perspectiva de ingeniería.
 - Diferenciar un LLM de un agente.
 - Entender por qué los agentes representan una evolución arquitectónica y no un nuevo tipo de modelo.
 - Identificar los componentes básicos de un sistema basado en agentes.
-

Introducción

Hasta este punto del libro analizamos cómo un Large Language Model interpreta instrucciones, genera texto y, mediante arquitecturas RAG, utiliza conocimiento externo para responder consultas fundamentadas.

Sin embargo, muchas aplicaciones empresariales requieren algo más que generar respuestas.

Un asistente puede explicar cómo crear un usuario.

Un agente puede crear ese usuario.

Un asistente puede describir el procedimiento para generar un informe.

Un agente puede consultar distintos sistemas, consolidar la información y entregar el informe terminado.

La diferencia ya no reside únicamente en comprender lenguaje.

Reside en actuar sobre el entorno.

El cambio de paradigma

Durante los primeros años de la IA generativa, la interacción seguía un patrón simple:

1. El usuario formula un objetivo.
2. El modelo interpreta la solicitud.
3. El sistema ejecuta una o más acciones.
4. El resultado se valida y se devuelve al usuario.

Los agentes incorporan capacidades de planificación, uso de herramientas, memoria y ejecución coordinada.

¿Qué es un agente?

Desde una perspectiva de ingeniería, un agente es un sistema capaz de perseguir un objetivo mediante la combinación de razonamiento, memoria, herramientas y capacidad de ejecución.

El modelo de lenguaje continúa siendo un componente fundamental.

Sin embargo, ya no constituye todo el sistema.

El agente incorpora una capa de orquestación que coordina el resto de los componentes.

Componentes principales

Una arquitectura basada en agentes suele incluir:

- un modelo de lenguaje;
- memoria;
- herramientas (*tools*);
- conectores hacia sistemas externos;
- un planificador;
- un orquestador;
- mecanismos de observabilidad;
- políticas de seguridad.

Cada componente posee una responsabilidad claramente definida.

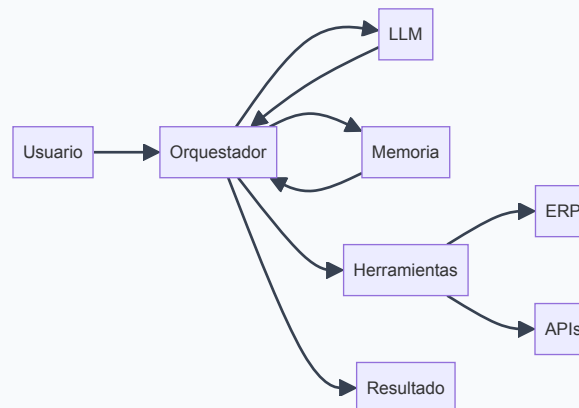
Un ejemplo

Solicitud:

"Prepará el informe mensual de ventas y enviáselo al director financiero."

Un agente podría:

1. Consultar el ERP.
2. Obtener las ventas del período.
3. Generar gráficos.
4. Redactar un resumen.
5. Enviar el correo.
6. Registrar la auditoría.



Ideas clave

- Un agente busca cumplir objetivos.

- El LLM es solo uno de sus componentes.
- Herramientas, memoria y planificación amplían sus capacidades.
- La arquitectura determina el comportamiento final.

Próxima sección

Analizaremos la arquitectura interna de un agente moderno y el ciclo completo de planificación y ejecución.

Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones.

Anatomía de un agente de IA

"La inteligencia de un agente no depende únicamente del modelo que utiliza, sino de cómo coordina todos sus componentes."

Objetivos de aprendizaje

Al finalizar esta sección podrás:

- Comprender los componentes internos de un agente moderno.
- Entender el flujo de planificación y ejecución.
- Diferenciar razonamiento, memoria, herramientas y orquestación.
- Identificar las responsabilidades arquitectónicas de cada componente.

Introducción

En la sección anterior definimos un agente como un sistema orientado a objetivos.

Ahora analizaremos su estructura interna.

Aunque distintas plataformas implementan agentes de formas diferentes, la mayoría comparte un conjunto de componentes fundamentales.

Comprender estas responsabilidades resulta mucho más importante que aprender una biblioteca específica.

Las herramientas evolucionan.

Los principios arquitectónicos permanecen.

El modelo de lenguaje

El Large Language Model continúa siendo el núcleo del razonamiento.

Su función consiste en:

- interpretar instrucciones;
- comprender el contexto;
- analizar información;
- generar planes;
- decidir qué acción realizar a continuación.

Sin embargo, el modelo no ejecuta directamente esas acciones.

Necesita colaborar con otros componentes.

El planificador

Una característica distintiva de los agentes es la capacidad de dividir un objetivo complejo en una secuencia de tareas.

Por ejemplo, ante la solicitud:

"Generá un informe de ventas del último trimestre y enviáselo al director comercial."

El planificador podría producir una secuencia como:

1. Consultar el ERP.
2. Obtener las ventas.
3. Consolidar los datos.
4. Generar gráficos.
5. Redactar el informe.
6. Enviar el correo.
7. Registrar la auditoría.

Esta planificación permite abordar problemas que no pueden resolverse con una única llamada al modelo.

La memoria

Un agente necesita recordar información mientras trabaja.

Dependiendo de la arquitectura, la memoria puede dividirse en diferentes niveles.

Memoria de corto plazo

Mantiene el contexto de la tarea actual.

Memoria de largo plazo

Conserva información útil entre distintas sesiones, como preferencias del usuario, configuraciones o resultados previamente validados.

No toda la información debe almacenarse en el contexto del LLM.

Una gestión adecuada de la memoria mejora el rendimiento y reduce costos.

Las herramientas

Los agentes amplían sus capacidades mediante herramientas externas.

Algunos ejemplos incluyen:

- consultar una base de datos;

- invocar una API;
- ejecutar código;
- acceder a un sistema ERP;
- enviar correos electrónicos;
- generar documentos;
- realizar búsquedas en Internet;
- interactuar con otros agentes.

Cada herramienta posee una interfaz bien definida.

El agente decide cuándo utilizarla y cómo interpretar sus resultados.

El orquestador

El orquestador coordina el funcionamiento completo del sistema.

Entre sus responsabilidades se encuentran:

- iniciar la ejecución;
- administrar el estado;
- invocar herramientas;
- controlar errores;
- registrar auditoría;
- aplicar políticas de seguridad;
- finalizar la tarea.

En sistemas empresariales, el orquestador suele contener una parte importante de la lógica de negocio.

Flujo general



Syntax error in text
mermaid version 10.9.6

Este flujo puede repetirse múltiples veces hasta completar el objetivo.

Caso de estudio

Una empresa desarrolla un agente para asistir al área de compras.

El usuario solicita:

"Compará los precios de los tres últimos proveedores y recomendá la mejor alternativa."

El agente no puede responder únicamente con el conocimiento del modelo.

Debe:

- consultar el sistema de compras;
- recuperar órdenes anteriores;
- comparar valores;
- aplicar criterios definidos por la organización;
- justificar la recomendación;
- registrar la decisión.

El modelo aporta razonamiento.

La arquitectura aporta capacidad operativa.

Ideas clave

- Un agente está compuesto por múltiples componentes especializados.
 - El LLM representa solo una parte del sistema.
 - Planificación, memoria, herramientas y orquestación trabajan de forma coordinada.
 - La calidad del agente depende tanto de la arquitectura como del modelo utilizado.
-

Próxima sección

En la siguiente sección estudiaremos el ciclo de razonamiento de un agente, analizando cómo planifica, ejecuta, observa resultados y adapta sus acciones hasta alcanzar el objetivo solicitado.

Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones.

El ciclo de razonamiento de un agente

"Un agente no ejecuta acciones al azar. Observa, razona, planifica, actúa y aprende del resultado obtenido."

Objetivos de aprendizaje

Al finalizar esta sección podrás:

- Comprender el ciclo operativo de un agente moderno.
 - Diferenciar razonamiento de ejecución.
 - Entender cómo un agente adapta su comportamiento durante una tarea.
 - Identificar los puntos donde un arquitecto puede intervenir para mejorar el sistema.
-

Introducción

En los capítulos anteriores analizamos sistemas cuyo comportamiento seguía un flujo relativamente lineal.

Un usuario realizaba una consulta.

El sistema recuperaba información.

El modelo generaba una respuesta.

En un agente, el comportamiento deja de ser lineal.

La ejecución se convierte en un proceso iterativo donde cada acción modifica el contexto de la siguiente.

El agente no solo responde.

Evalúa continuamente si está avanzando hacia el objetivo.

Un ciclo continuo

Aunque existen distintas implementaciones, la mayoría de los agentes sigue un patrón similar.

1. Comprender el objetivo.
2. Elaborar un plan.
3. Seleccionar una acción.
4. Ejecutar la acción.
5. Observar el resultado.
6. Evaluar si el objetivo fue alcanzado.
7. Repetir el ciclo cuando sea necesario.

Este mecanismo permite resolver problemas cuya solución no puede determinarse en una única inferencia.

Comprender antes de actuar

Toda ejecución comienza interpretando correctamente la solicitud.

Una misma instrucción puede admitir múltiples estrategias.

Por ejemplo:

"Preparará un informe de ventas."

El agente deberá responder preguntas como:

- ¿De qué período?
- ¿Qué sistema contiene la información?
- ¿Debe incluir gráficos?
- ¿Quién recibirá el informe?
- ¿Existe un formato corporativo?

Responder estas cuestiones forma parte del razonamiento inicial.

Planificación

Una vez comprendido el objetivo, el agente construye un plan.

El plan no tiene por qué ser definitivo.

Puede modificarse durante la ejecución.

Este comportamiento diferencia a un agente de un flujo rígido preprogramado.

El sistema adapta sus decisiones según la información obtenida en cada paso.

Acción

Cada acción implica utilizar una herramienta.

Algunos ejemplos incluyen:

- ejecutar una consulta SQL;
- llamar a una API REST;
- consultar un ERP;

- buscar documentos mediante RAG;
- enviar un correo electrónico;
- ejecutar un script.

El modelo no realiza directamente estas operaciones.

Las solicita al orquestador mediante interfaces previamente definidas.

Observación

Después de ejecutar una acción, el agente analiza el resultado.

Puede descubrir que:

- obtuvo la información esperada;
- la consulta produjo un error;
- faltan datos;
- es necesario consultar otra fuente;
- el plan debe modificarse.

Esta retroalimentación convierte al agente en un sistema adaptativo.

Reflexión

Las arquitecturas más avanzadas incorporan una etapa adicional.

El agente evalúa la calidad de su propio trabajo antes de finalizar.

Puede verificar si:

- respondió completamente;
- utilizó información suficiente;
- existen inconsistencias;
- conviene ejecutar pasos adicionales.

Este proceso suele conocerse como *reflection* o *self-evaluation*.

No implica conciencia.

Representa un mecanismo de control diseñado para incrementar la calidad del resultado.

Flujo completo



Syntax error in text
mermaid version 10.9.6

Obsérvese que el ciclo puede repetirse múltiples veces.

Caso de estudio

Una empresa solicita a un agente:

"Analizá las ventas del último semestre, identificá anomalías y prepará recomendaciones."

Durante la ejecución ocurre lo siguiente.

1. Consulta el sistema comercial.
2. Detecta datos incompletos.
3. Consulta una segunda fuente.
4. Recalcula los indicadores.
5. Genera gráficos.
6. Elabora conclusiones.
7. Verifica consistencia.
8. Produce el informe final.

El flujo no estaba completamente definido desde el principio.

Fue adaptándose según los resultados obtenidos.

Desafíos arquitectónicos

Implementar este ciclo implica resolver varios problemas.

- Evitar bucles infinitos.
- Limitar el número de acciones.
- Administrar tiempos de espera.
- Gestionar errores de herramientas.
- Registrar todas las decisiones.
- Controlar costos.
- Mantener trazabilidad.

Estas responsabilidades pertenecen a la arquitectura y no al modelo.

Ideas clave

- Un agente ejecuta un ciclo iterativo de razonamiento y acción.
 - Cada resultado modifica el contexto de la siguiente decisión.
 - La reflexión permite mejorar la calidad antes de finalizar.
 - El orquestador controla la ejecución y evita comportamientos no deseados.
-

Próxima sección

En la siguiente sección estudiaremos las herramientas (*Tools*) y el mecanismo de *Function Calling*, comprendiendo cómo un agente interactúa de forma segura con APIs, bases de datos y aplicaciones empresariales.

Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones.

Tools y Function Calling: cómo un agente interactúa con el mundo

"La diferencia entre un chatbot y un agente suele comenzar cuando el modelo deja de responder únicamente con texto y empieza a utilizar herramientas."

Objetivos de aprendizaje

Al finalizar esta sección podrás:

- Comprender qué es una *tool* dentro de una arquitectura de agentes.
- Entender el concepto de *Function Calling*.
- Diferenciar generación de texto de ejecución de acciones.
- Diseñar interfaces seguras entre un agente y sistemas empresariales.

Introducción

Un Large Language Model posee una extraordinaria capacidad para interpretar lenguaje natural y generar respuestas.

Sin embargo, no puede consultar una base de datos corporativa, enviar un correo electrónico ni crear un usuario en un ERP por sí mismo.

Para realizar esas acciones necesita un mecanismo de integración.

Ese mecanismo se materializa mediante **tools** y **Function Calling**.

Lejos de ser una característica secundaria, constituyen uno de los pilares de la Ingeniería de IA moderna.

¿Qué es una tool?

Una *tool* es una capacidad externa que el agente puede invocar durante la resolución de una tarea.

Puede representar prácticamente cualquier operación accesible mediante una interfaz bien definida.

Algunos ejemplos son:

- consultar una API REST;
- ejecutar una consulta SQL;
- buscar información en un sistema RAG;
- enviar un correo electrónico;
- crear un ticket;
- generar un documento PDF;
- ejecutar código controlado;
- invocar otro agente.

Desde la perspectiva del modelo, todas estas operaciones se presentan como funciones disponibles.

¿Qué es Function Calling?

El *Function Calling* permite que el modelo indique **qué herramienta desea utilizar y con qué parámetros**, en lugar de intentar ejecutar la acción directamente.

Este cambio es fundamental.

El modelo no accede de manera autónoma a un sistema externo.

Genera una intención estructurada.

El orquestador interpreta esa intención, valida los parámetros y decide si la operación puede ejecutarse.

De esta forma, el modelo propone acciones, mientras que la arquitectura conserva el control.

Separación de responsabilidades

Una arquitectura madura distingue claramente tres responsabilidades.

El modelo

- interpreta la solicitud;
- decide qué herramienta necesita.

El orquestador

- valida la solicitud;
- aplica políticas de seguridad;
- invoca la herramienta correspondiente.

La herramienta

- ejecuta la operación;
- devuelve un resultado estructurado.

Esta separación reduce el acoplamiento y facilita el reemplazo de cualquiera de los componentes.

Flujo de ejecución



Syntax error in text
mermaid version 10.9.6

Obsérvese que el modelo nunca interactúa directamente con los sistemas corporativos.

Siempre existe una capa de control.

Diseño de herramientas

Una herramienta bien diseñada debería cumplir varios principios.

Responsabilidad única

Cada herramienta debe realizar una tarea claramente definida.

Contratos explícitos

Los parámetros de entrada y salida deben estar completamente especificados.

Idempotencia cuando sea posible

Ejecutar la misma operación varias veces no debería producir efectos inesperados.

Manejo de errores

Toda herramienta debe devolver errores estructurados que el agente pueda interpretar.

Auditoría

Cada invocación debería quedar registrada para facilitar trazabilidad y cumplimiento normativo.

Seguridad

Uno de los errores más peligrosos consiste en otorgar al modelo acceso irrestricto a sistemas críticos.

La arquitectura debe incorporar mecanismos como:

- autenticación;
- autorización;
- validación de parámetros;
- listas de operaciones permitidas;
- límites de ejecución;
- confirmación humana para acciones críticas.

Un agente nunca debería ejecutar operaciones sensibles únicamente porque un usuario las solicitó en lenguaje natural.

Caso de estudio

Un agente de soporte recibe la solicitud:

"Deshabilita el acceso VPN del usuario Juan Pérez."

El modelo identifica que debe utilizar la herramienta **DeshabilitarVPN()**.

Sin embargo, antes de ejecutar la acción, el orquestador verifica:

- que el usuario solicitante posee permisos;
- que la operación requiere aprobación;
- que el identificador corresponde a un usuario existente;
- que la auditoría está habilitada.

Solo entonces la herramienta realiza la modificación.

La inteligencia del modelo no reemplaza las políticas de seguridad de la organización.

Las complementa.

Buenas prácticas

- Mantener herramientas pequeñas y especializadas.
- Evitar exponer operaciones innecesarias al modelo.
- Versionar las interfaces.
- Registrar todas las ejecuciones.
- Separar claramente razonamiento y ejecución.
- Validar siempre los parámetros antes de invocar sistemas externos.

Ideas clave

- Una *tool* amplía las capacidades del agente.
 - El *Function Calling* no ejecuta acciones; solicita su ejecución.
 - El orquestador mantiene el control sobre las operaciones.
 - Seguridad, auditoría y contratos son responsabilidades arquitectónicas.
-

Próxima sección

En la siguiente sección estudiaremos la memoria de los agentes, analizando cómo administran contexto, conocimiento persistente y experiencias previas para resolver tareas cada vez más complejas.

Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones.

La memoria de los agentes: recordar para actuar mejor

"Un agente sin memoria debe redescubrir el mundo en cada interacción. Un agente con memoria puede construir experiencia."

Objetivos de aprendizaje

Al finalizar esta sección podrás:

- Comprender por qué la memoria es un componente esencial de un agente.
- Diferenciar memoria de trabajo, memoria a largo plazo, memoria semántica y memoria episódica.
- Entender cómo se implementan estos conceptos en arquitecturas empresariales.
- Diseñar estrategias de memoria escalables y seguras.

Introducción

Hasta este punto analizamos cómo un agente razona, planifica y utiliza herramientas.

Sin embargo, existe una capacidad adicional que determina gran parte de su utilidad: la memoria.

Sin memoria, cada solicitud constituye un problema completamente nuevo.

El agente olvida conversaciones anteriores, decisiones tomadas y resultados obtenidos.

En aplicaciones reales esto resulta inaceptable.

Los usuarios esperan continuidad, personalización y aprendizaje a lo largo del tiempo.

La memoria permite construir esa continuidad.

¿Qué entendemos por memoria?

En el contexto de la Ingeniería de IA, la memoria es el conjunto de mecanismos que permiten conservar información útil para tareas presentes o futuras.

No toda la información merece el mismo tratamiento.

Por ello, la mayoría de las arquitecturas distingue distintos tipos de memoria según su duración, propósito y forma de recuperación.

Memoria de trabajo

La memoria de trabajo contiene únicamente la información necesaria para resolver la tarea actual.

Incluye elementos como:

- el objetivo solicitado;
- el contexto de la conversación;
- resultados intermedios;
- estado de las herramientas utilizadas.

Su vida útil suele finalizar cuando termina la ejecución.

Mantener esta memoria pequeña mejora el rendimiento y reduce el consumo de contexto del LLM.

Memoria a largo plazo

La memoria a largo plazo conserva información entre distintas sesiones.

Puede almacenar:

- preferencias del usuario;
- configuraciones habituales;
- historial de decisiones;
- conocimiento específico del dominio;
- resultados previamente validados.

En la práctica, esta memoria suele implementarse mediante bases de datos, repositorios documentales o sistemas RAG.

El LLM no almacena directamente esta información.

La recupera cuando resulta necesaria.

Memoria semántica

La memoria semántica representa conocimiento relativamente estable.

Por ejemplo:

- documentación técnica;
- políticas corporativas;
- manuales;
- procedimientos;
- glosarios;
- normas de negocio.

Generalmente se implementa mediante repositorios indexados y búsqueda semántica.

Cuando el agente necesita responder una consulta, recupera únicamente los fragmentos relevantes.

Memoria episódica

La memoria episódica registra experiencias concretas.

Algunos ejemplos incluyen:

- tareas ejecutadas anteriormente;
- errores detectados;
- decisiones tomadas;
- resultados obtenidos;
- conversaciones relevantes.

Esta información permite evitar repetir errores y facilita auditorías posteriores.

En ciertos escenarios también puede utilizarse para mejorar futuras planificaciones.

Arquitectura de memoria

Una implementación típica puede representarse de la siguiente manera:



Syntax error in text
mermaid version 10.9.6

Cada tipo de memoria cumple una función diferente.

No existe una única memoria universal.

¿Debe recordarse todo?

No.

Recordar indiscriminadamente produce problemas.

Entre ellos:

- crecimiento innecesario del contexto;
- incremento de costos;
- información obsoleta;
- riesgos de privacidad;
- respuestas inconsistentes.

Un arquitecto debe definir políticas claras sobre:

- qué recordar;
- durante cuánto tiempo;
- quién puede acceder;
- cuándo eliminar la información.

La memoria también forma parte del gobierno de datos.

Caso de estudio

Una mesa de ayuda incorpora un agente para asistir a los técnicos.

Durante la primera versión, cada conversación comenzaba desde cero.

Los usuarios debían repetir constantemente información sobre el sistema afectado, el historial del incidente y las acciones ya realizadas.

En una segunda versión se añadió memoria episódica.

El agente recuperaba automáticamente los eventos relacionados con el ticket actual y utilizaba esa información para continuar el diagnóstico.

El cambio no mejoró el modelo.

Mejóro la arquitectura de memoria.

Buenas prácticas

- Separar memoria temporal de memoria persistente.
- No almacenar información sensible sin controles adecuados.

- Aplicar políticas de expiración y versionado.
- Recuperar únicamente la memoria relevante para cada tarea.
- Auditar el acceso a la información persistente.
- Diseñar la memoria como un servicio independiente del LLM.

Ideas clave

- La memoria permite continuidad entre interacciones.
- Existen distintos tipos de memoria con responsabilidades diferentes.
- La memoria persistente suele implementarse fuera del modelo.
- Gestionar correctamente la memoria mejora precisión, costos y experiencia del usuario.

Próxima sección

En la siguiente sección estudiaremos los sistemas multiagente, comprendiendo cómo varios agentes especializados pueden colaborar para resolver problemas que exceden las capacidades de un único agente.

Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones.

Sistemas Multiagente: inteligencia distribuida

"Un agente puede resolver muchas tareas. Un conjunto de agentes especializados puede resolver problemas cuya complejidad supera las capacidades de un único componente."

Objetivos de aprendizaje

Al finalizar esta sección podrás:

- Comprender qué es un sistema multiagente.
 - Diferenciar un agente generalista de un conjunto de agentes especializados.
 - Entender los patrones de coordinación más utilizados.
 - Identificar cuándo conviene adoptar una arquitectura multiagente.
-

Introducción

A medida que las aplicaciones de IA crecen, también aumenta la complejidad de las tareas que deben resolver.

Un único agente puede transformarse rápidamente en un cuello de botella.

Debe conocer demasiados procesos, utilizar numerosas herramientas y tomar decisiones sobre dominios muy diferentes.

Una alternativa consiste en distribuir esas responsabilidades.

En lugar de construir un agente capaz de hacerlo todo, se diseñan varios agentes especializados que colaboran entre sí.

Este enfoque da origen a los **Sistemas Multiagente (Multi-Agent Systems, MAS)**.

¿Por qué varios agentes?

La división de responsabilidades aporta ventajas similares a las observadas en la ingeniería de software.

Cada agente puede especializarse en un dominio concreto.

Por ejemplo:

- agente de búsqueda documental;
- agente financiero;
- agente legal;

- agente de soporte técnico;
- agente de planificación;
- agente de auditoría.

Esta especialización simplifica la evolución del sistema y mejora la calidad de las decisiones dentro de cada dominio.

Patrones de coordinación

Existen múltiples estrategias para coordinar agentes.

Orquestación

Un agente coordinador recibe la solicitud, divide el trabajo y distribuye tareas entre los agentes especializados.

Este patrón facilita el control, la auditoría y la aplicación de políticas.

Coreografía

Los agentes colaboran directamente entre sí mediante eventos o mensajes, sin depender de un coordinador central.

Este enfoque favorece el desacoplamiento, aunque incrementa la complejidad de diseño.

La elección depende de los requisitos de cada organización.

Comunicación entre agentes

La colaboración requiere un lenguaje común.

En la práctica, los agentes intercambian información mediante:

- mensajes estructurados;
- APIs;
- colas de eventos;
- llamadas a herramientas;

- documentos compartidos.

Es recomendable que la comunicación utilice contratos explícitos y formatos bien definidos para facilitar la interoperabilidad.

Evitando duplicación de responsabilidades

Un error frecuente consiste en crear múltiples agentes con funciones similares.

Esto genera:

- decisiones inconsistentes;
- mayor consumo de recursos;
- mantenimiento complejo;
- dificultades para el gobierno del sistema.

Cada agente debe poseer un propósito claramente delimitado y responsabilidades bien definidas.

Arquitectura de referencia



Syntax error in text
mermaid version 10.9.6

El usuario interactúa con un único punto de entrada, mientras que la complejidad interna permanece encapsulada.

Caso de estudio

Una empresa desea automatizar el proceso de incorporación de nuevos empleados.

El agente orquestador recibe la solicitud y distribuye tareas.

- El agente de RR. HH. valida la información del colaborador.
- El agente de infraestructura crea las cuentas necesarias.
- El agente de seguridad asigna permisos.
- El agente documental genera contratos y formularios.
- El agente de auditoría registra todas las acciones realizadas.

Cada componente trabaja sobre su propio dominio, mientras el orquestador consolida el resultado final.

Desafíos

Las arquitecturas multiagente introducen nuevos problemas.

- Coordinación entre agentes.
- Gestión de estados compartidos.
- Resolución de conflictos.
- Tolerancia a fallos.
- Observabilidad distribuida.
- Control de costos.
- Latencia acumulada.

Por este motivo, no toda aplicación requiere múltiples agentes.

La complejidad debe justificarse mediante necesidades reales del negocio.

Buenas prácticas

- Definir responsabilidades claras.
 - Mantener interfaces estables entre agentes.
 - Evitar dependencias circulares.
 - Centralizar observabilidad y auditoría.
 - Medir el valor aportado por cada agente.
 - Incorporar mecanismos de recuperación ante fallos.
-

Ideas clave

- Un sistema multiagente distribuye responsabilidades entre componentes especializados.
 - La coordinación puede realizarse mediante orquestación o coreografía.
 - La especialización mejora mantenibilidad y escalabilidad.
 - La complejidad adicional debe responder a una necesidad arquitectónica concreta.
-

Próxima sección

En la siguiente sección analizaremos la planificación avanzada (*Planning*), la descomposición automática de objetivos y las estrategias utilizadas por los agentes para resolver tareas complejas de múltiples pasos.

Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones.

Planning: cómo un agente transforma objetivos en planes ejecutables

"Resolver un problema complejo rara vez consiste en ejecutar una única acción. La clave está en construir un plan capaz de adaptarse al contexto."

Objetivos de aprendizaje

Al finalizar esta sección podrás:

- Comprender el papel de la planificación dentro de un agente.
- Diferenciar un objetivo de un plan de ejecución.
- Conocer estrategias para descomponer tareas complejas.
- Incorporar criterios arquitectónicos para diseñar agentes capaces de planificar.

Introducción

En una aplicación tradicional, el flujo de ejecución suele estar definido por el desarrollador.

El sistema conoce de antemano qué pasos debe seguir.

Los agentes modifican este paradigma.

Reciben un objetivo de alto nivel y deben decidir por sí mismos qué acciones ejecutar, en qué orden y con qué herramientas.

La planificación se convierte así en una capacidad fundamental.

Del objetivo al plan

Los usuarios rara vez describen todos los pasos necesarios para completar una tarea.

Una solicitud como:

"Prepará la documentación para incorporar un nuevo empleado."

encierra múltiples actividades implícitas.

El agente debe identificar dichas actividades y organizarlas en una secuencia coherente.

Entre ellas podrían encontrarse:

- validar la identidad;
- crear cuentas de acceso;
- asignar permisos;
- generar documentación;
- notificar a las áreas involucradas.

La planificación traduce una intención general en acciones concretas.

Descomposición de tareas

Uno de los principios más importantes consiste en dividir problemas complejos en subtareas manejables.

Este enfoque ofrece varias ventajas:

- facilita la reutilización de herramientas;
- simplifica la recuperación ante errores;
- permite ejecutar tareas en paralelo cuando corresponda;
- mejora la observabilidad;
- reduce el riesgo de decisiones incorrectas.

La descomposición también favorece la colaboración entre agentes especializados.

Planes dinámicos

No todos los planes permanecen estables.

Durante la ejecución pueden surgir eventos inesperados.

Por ejemplo:

- una API deja de responder;
- un documento no existe;
- un usuario carece de permisos;
- una herramienta devuelve información incompleta.

Un agente maduro no continúa ejecutando el plan original sin modificaciones.

Reevalúa la situación y genera un nuevo plan teniendo en cuenta el estado actual del sistema.

Esta capacidad convierte la planificación en un proceso continuo y no en una etapa aislada.

Dependencias

Las tareas rara vez son independientes.

Algunas requieren que otras hayan finalizado previamente.

Por ejemplo, no es posible asignar permisos a una cuenta que todavía no fue creada.

El planificador debe identificar estas relaciones para evitar errores de ejecución.

Desde una perspectiva arquitectónica, esto implica modelar dependencias explícitas entre acciones.

Priorización

Cuando existen múltiples tareas posibles, el agente necesita establecer prioridades.

Algunos criterios habituales incluyen:

- criticidad para el negocio;
- disponibilidad de recursos;
- tiempo estimado de ejecución;
- impacto sobre el usuario;
- riesgo asociado.

La priorización permite optimizar tanto el rendimiento como la experiencia final.

Flujo de planificación



Syntax error in text
mermaid version 10.9.6

El ciclo refleja que la planificación no finaliza cuando comienza la ejecución.

Ambos procesos evolucionan conjuntamente.

Caso de estudio

Una empresa solicita al agente:

"Migra el sistema de desarrollo al nuevo entorno."

Durante el análisis inicial el agente identifica decenas de tareas.

Al comenzar la ejecución descubre que uno de los servidores no se encuentra disponible.

En lugar de cancelar toda la operación, reorganiza el plan.

Ejecuta las tareas independientes, registra la incidencia y pospone aquellas que dependen del servidor afectado.

La capacidad de replantear el plan evita interrupciones innecesarias y mejora la resiliencia del proceso.

Buenas prácticas

- Diseñar tareas pequeñas y reutilizables.
- Expresar dependencias de forma explícita.
- Permitir replanificación durante la ejecución.
- Registrar cada decisión tomada por el planificador.
- Establecer límites para evitar ciclos infinitos.
- Medir el desempeño del plan y no solo el resultado final.

Ideas clave

- Un objetivo no constituye un plan.
 - La planificación transforma intenciones en acciones concretas.
 - Los planes deben adaptarse a los cambios del entorno.
 - La calidad del planificador influye directamente sobre la eficacia del agente.
-

Próxima sección

En la siguiente sección estudiaremos la observabilidad y el gobierno de agentes de IA, analizando cómo auditar decisiones, controlar costos, registrar ejecuciones y garantizar un funcionamiento seguro en entornos empresariales.

Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones.

Observabilidad, Gobierno y Seguridad en Sistemas de Agentes

"La diferencia entre un experimento y una plataforma empresarial no está en la calidad del modelo, sino en la capacidad de gobernar su comportamiento."

Objetivos de aprendizaje

Al finalizar esta sección podrás:

- Comprender la importancia de la observabilidad en arquitecturas basadas en agentes.
 - Identificar los principales mecanismos de gobierno y control.
 - Entender el papel del enfoque Human-in-the-Loop (HITL).
 - Diseñar sistemas auditables, seguros y preparados para producción.
-

Introducción

Construir un agente capaz de ejecutar tareas complejas representa un desafío técnico importante.

Sin embargo, desplegar ese mismo agente dentro de una organización incorpora nuevas exigencias.

La empresa necesita responder preguntas como:

- ¿Qué decisión tomó el agente?
- ¿Qué información utilizó?
- ¿Qué herramientas ejecutó?
- ¿Quién autorizó la operación?
- ¿Cuál fue el costo?
- ¿Cómo reconstruir una ejecución ocurrida hace seis meses?

Responder estas preguntas requiere mucho más que un buen modelo.

Requiere una arquitectura de gobierno.

Observabilidad

La observabilidad consiste en obtener información suficiente para comprender el comportamiento interno del sistema.

En una arquitectura basada en agentes conviene registrar, al menos:

- objetivo solicitado;
- plan generado;
- herramientas utilizadas;
- parámetros enviados;
- respuestas obtenidas;
- duración de cada etapa;
- cantidad de tokens consumidos;
- costo estimado;
- resultado final.

Estos registros permiten diagnosticar problemas y optimizar el sistema de manera continua.

Auditoría

No toda la información registrada tiene únicamente fines técnicos.

En muchos sectores existen requisitos regulatorios.

Por ejemplo:

- administración pública;
- salud;
- sistema financiero;
- justicia;
- infraestructura crítica.

En estos escenarios es indispensable reconstruir exactamente qué ocurrió durante una ejecución.

La auditoría proporciona esa capacidad.

Gobierno

El gobierno establece las reglas bajo las cuales puede operar un agente.

Algunos ejemplos incluyen:

- herramientas autorizadas;
- operaciones prohibidas;
- límites de gasto;
- horarios permitidos;
- clasificación de información;
- políticas de retención;
- segregación de funciones.

Estas reglas no deberían implementarse dentro del prompt.

Deben formar parte de la arquitectura.

Human-in-the-Loop

No todas las decisiones deben automatizarse completamente.

En operaciones críticas resulta habitual incorporar un mecanismo de aprobación humana.

Este enfoque se conoce como **Human-in-the-Loop (HITL)**.

El agente puede preparar una propuesta, pero la ejecución definitiva requiere validación por parte de una persona autorizada.

Algunos ejemplos son:

- aprobar una transferencia bancaria;
- eliminar información sensible;
- otorgar permisos privilegiados;
- firmar documentación legal;
- publicar cambios en producción.

Gestión de riesgos

Toda arquitectura basada en agentes debe considerar riesgos como:

- ejecución de acciones incorrectas;
- acceso indebido a información;
- uso excesivo de recursos;
- errores de herramientas externas;
- respuestas inconsistentes;
- ataques mediante prompt injection;
- fuga de información confidencial.

La mitigación de estos riesgos requiere controles en múltiples capas.

Arquitectura de control



Syntax error in text
mermaid version 10.9.6

Obsérvese que la seguridad no depende exclusivamente del modelo.

Está distribuida entre distintos componentes.

Caso de estudio

Una organización implementa un agente con capacidad para administrar usuarios corporativos.

Durante una prueba, un empleado solicita:

"Eliminá todas las cuentas inactivas."

El agente identifica correctamente la herramienta necesaria.

Sin embargo, las políticas establecen que cualquier eliminación masiva requiere aprobación de un administrador.

El orquestador detiene la ejecución, genera una solicitud de aprobación y registra toda la operación.

La seguridad no provino del modelo.

Provino de la arquitectura.

Buenas prácticas

- Registrar todas las ejecuciones relevantes.
 - Aplicar políticas fuera del modelo.
 - Incorporar aprobación humana en operaciones críticas.
 - Medir costos y consumo de recursos.
 - Centralizar observabilidad.
 - Diseñar mecanismos de recuperación ante fallos.
 - Revisar periódicamente permisos y herramientas disponibles.
-

Ideas clave

- Observabilidad, auditoría y gobierno son componentes esenciales de una plataforma de agentes.
 - Las políticas deben implementarse en la arquitectura y no depender del prompt.
 - Human-in-the-Loop reduce riesgos en operaciones sensibles.
 - Un sistema empresarial debe ser explicable, auditable y controlable.
-

Próxima sección

En la siguiente sección estudiaremos patrones de diseño para agentes empresariales, analizando arquitecturas desacopladas, integración con microservicios, resiliencia y escalabilidad.

Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones.

Patrones arquitectónicos para agentes empresariales

"Un agente útil resuelve una tarea. Una arquitectura de agentes bien diseñada permite que cientos de ellos evolucionen sin perder control, seguridad ni mantenibilidad."

Objetivos de aprendizaje

Al finalizar esta sección podrás:

- Comprender los principales patrones arquitectónicos para agentes empresariales.
 - Diseñar soluciones desacopladas y escalables.
 - Integrar agentes con microservicios, eventos y sistemas existentes.
 - Identificar decisiones arquitectónicas que favorecen la resiliencia y el mantenimiento.
-

Introducción

Los primeros prototipos de agentes suelen concentrar toda la lógica en un único proceso.

El mismo componente recibe la solicitud, razona, invoca herramientas, almacena memoria y devuelve una respuesta.

Este enfoque resulta adecuado para validar una idea.

Sin embargo, cuando la solución comienza a crecer aparecen problemas de escalabilidad, mantenimiento y gobierno.

La arquitectura debe evolucionar.

Desacoplamiento

Una plataforma empresarial debería separar claramente sus responsabilidades.

Una arquitectura habitual incluye:

- interfaz de usuario;
- API de entrada;
- orquestador de agentes;
- servicio de memoria;
- servicio RAG;
- catálogo de herramientas;
- observabilidad;
- auditoría.

Cada componente puede evolucionar independientemente.

Esta separación reduce el impacto de los cambios y facilita las pruebas.

Integración con microservicios

En muchas organizaciones los sistemas ya existen.

ERP, CRM, RR. HH., autenticación, facturación y otros servicios forman parte de la plataforma tecnológica.

El agente no debería reemplazarlos.

Debe integrarse con ellos mediante contratos estables.

En este contexto, los agentes se comportan como consumidores de capacidades ya disponibles.

La lógica de negocio permanece donde corresponde.

Arquitecturas orientadas a eventos

No todas las interacciones requieren una respuesta inmediata.

Algunas tareas pueden ejecutarse de forma asincrónica.

Por ejemplo:

- generar informes extensos;
- procesar grandes volúmenes documentales;
- ejecutar migraciones;
- enviar notificaciones masivas.

En estos escenarios resulta conveniente utilizar eventos y colas de mensajes.

El agente inicia el proceso y continúa con otras tareas mientras la operación se ejecuta en segundo plano.

Resiliencia

Toda arquitectura debe asumir que los errores ocurrirán.

Una API puede dejar de responder.

Una herramienta puede exceder el tiempo máximo permitido.

Una base vectorial puede encontrarse temporalmente fuera de servicio.

Por ello conviene incorporar mecanismos como:

- reintentos controlados;
- tiempos de espera (*timeouts*);
- circuit breakers;
- degradación controlada;
- colas de recuperación;
- compensación de operaciones.

La resiliencia no elimina los errores.

Reduce su impacto.

Escalabilidad

Los distintos componentes presentan necesidades diferentes.

El servicio RAG puede requerir más memoria.

El LLM puede necesitar GPU.

El catálogo de herramientas puede crecer sin afectar al resto del sistema.

Diseñar servicios independientes permite escalar únicamente los componentes que realmente lo necesitan.

Arquitectura de referencia



Syntax error in text
mermaid version 10.9.6

Este diseño favorece el desacoplamiento y permite evolucionar cada servicio de forma independiente.

Caso de estudio

Una empresa comienza con un único agente encargado de atención interna.

Con el tiempo incorpora automatizaciones para recursos humanos, finanzas, infraestructura y compras.

En lugar de ampliar indefinidamente el agente original, adopta una arquitectura modular.

Cada dominio dispone de sus propias herramientas y reglas de negocio.

El orquestador decide qué agente o servicio debe intervenir en cada solicitud.

El resultado es un sistema más mantenible y preparado para crecer.

Buenas prácticas

- Mantener responsabilidades claramente separadas.
 - Reutilizar servicios existentes antes de duplicar lógica.
 - Diseñar contratos estables entre componentes.
 - Incorporar resiliencia desde el inicio.
 - Escalar servicios de forma independiente.
 - Centralizar monitoreo, métricas y auditoría.
-

Ideas clave

- Una arquitectura empresarial debe evolucionar más allá del prototipo inicial.
 - Los agentes deben integrarse con la plataforma existente, no reemplazarla.
 - Eventos, microservicios y desacoplamiento facilitan la evolución del sistema.
 - Escalabilidad y resiliencia forman parte del diseño arquitectónico.
-

Próxima sección

En la última sección del capítulo integraremos todos los conceptos estudiados sobre agentes de IA y construiremos una arquitectura de referencia completa, preparando el camino para los capítulos dedicados a plataformas de IA y MLOps.

Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones.

Integrando la arquitectura de agentes: de la conversación a la automatización inteligente

"Un agente no reemplaza una aplicación. La transforma en un sistema capaz de comprender objetivos, coordinar acciones y colaborar con el resto de la plataforma."

Objetivos de aprendizaje

Al finalizar esta sección podrás:

- Integrar todos los conceptos estudiados en este capítulo.
 - Comprender cómo construir una arquitectura empresarial basada en agentes.
 - Identificar los principios que distinguen un prototipo de una plataforma de producción.
 - Prepararte para el estudio de plataformas de IA, LLMOps y MLOps.
-

Introducción

A lo largo de este capítulo recorrimos la evolución desde un modelo de lenguaje hasta una arquitectura de agentes completa.

Analizamos cómo un agente interpreta objetivos, planifica, utiliza herramientas, administra memoria, coordina otros agentes y opera bajo políticas de gobierno.

Cada uno de estos componentes resuelve un problema específico.

Juntos conforman una plataforma capaz de ejecutar procesos complejos con un nivel de autonomía controlado.

El recorrido de una solicitud

Una petición aparentemente sencilla puede activar numerosos componentes.

Un flujo típico es el siguiente:

1. El usuario expresa un objetivo.
2. El orquestador interpreta la intención.
3. El planificador construye un plan.
4. Se recupera contexto desde memoria o RAG.
5. El agente selecciona herramientas.
6. Se ejecutan acciones sobre sistemas externos.
7. Los resultados se validan.
8. Se registra la auditoría.
9. Se entrega la respuesta.

La generación de texto representa solo una parte del proceso.



Syntax error in text
mermaid version 10.9.6

Principios arquitectónicos

Una plataforma de agentes preparada para producción suele compartir varios principios.

Separación de responsabilidades

El modelo razona.

El orquestador coordina.

Las herramientas ejecutan.

La memoria conserva conocimiento.

Cada componente posee un propósito claramente definido.

Gobierno centralizado

Las políticas de seguridad, permisos y auditoría no dependen del prompt.

Se implementan como capacidades de la plataforma.

Observabilidad completa

Cada decisión importante puede reconstruirse posteriormente.

Esto facilita mantenimiento, mejora continua y cumplimiento normativo.

Evolución independiente

El reemplazo de un modelo, una herramienta o una base vectorial no debería requerir rediseñar toda la arquitectura.

Errores frecuentes

Las implementaciones iniciales suelen presentar algunos patrones repetitivos.

- Concentrar toda la lógica en el LLM.
- Exponer herramientas sin controles.
- Carecer de auditoría.
- Mantener memoria ilimitada.
- Ignorar la observabilidad.
- Diseñar agentes monolíticos difíciles de evolucionar.

Estos problemas aparecen con frecuencia cuando se piensa en agentes como una funcionalidad y no como una arquitectura.

Caso de estudio

Una organización comienza desarrollando un asistente para responder preguntas internas.

Con el tiempo incorpora búsqueda documental, generación de informes, integración con el ERP, automatización de aprobaciones y coordinación entre distintos dominios.

El modelo utilizado apenas cambia.

Lo que evoluciona es la arquitectura que lo rodea.

Este patrón se observa repetidamente en proyectos exitosos de Ingeniería de IA.

Resumen del capítulo

Durante este capítulo aprendimos que:

- un agente persigue objetivos y no solo conversaciones;
 - herramientas y *Function Calling* amplían las capacidades del modelo;
 - la memoria permite continuidad y aprendizaje operativo;
 - múltiples agentes pueden colaborar mediante patrones de coordinación;
 - la planificación transforma objetivos en acciones ejecutables;
 - observabilidad, gobierno y seguridad son requisitos de primer nivel;
 - la arquitectura determina la capacidad de evolucionar una plataforma.
-

Lo que viene a continuación

Hasta aquí analizamos cómo construir sistemas inteligentes capaces de ejecutar tareas.

El siguiente desafío consiste en operarlos de forma profesional.

¿Cómo desplegar modelos?

¿Cómo versionarlos?

¿Cómo monitorear costos y rendimiento?

¿Cómo administrar prompts, modelos, datasets y pipelines a escala?

Estas preguntas introducen el siguiente gran tema del libro:

Plataformas de IA, LLMOps y MLOps.

Allí estudiaremos cómo transformar soluciones aisladas en plataformas empresariales sostenibles.

Mensaje final

La Ingeniería de IA no consiste únicamente en elegir el mejor modelo disponible.

Consiste en diseñar sistemas capaces de integrar modelos, datos, herramientas, memoria, procesos y personas dentro de una arquitectura gobernable.

Los agentes representan un paso decisivo hacia esa visión.

Comprender su funcionamiento permite construir plataformas que no solo responden preguntas, sino que colaboran activamente con la organización para alcanzar objetivos de negocio.

Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones.